Subject:	OOP-JAVA
Name:	Deep Dhakecha
ID No.:	25AIML006

Practical-4

Part A

PROBLEM-1 : Password strength (PasswordChecker.java + Driver.java)

(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.
(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4).
(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.
Expected output: "abc" → Weak; "Abcd1234!" → Strong.

CODE : PasswordChecker.java

```
public class PasswordChecker {
   public static boolean hasMinLength(String password) {
      return password.length() >= 8;
   }

   public static boolean hasUppercase(String password) {
      for (char c : password.toCharArray()) {
         if (Character.isUpperCase(c)) {
            return true;
```

```
        }
    }
    return false;
}

public static boolean hasDigit(String password) {
    for (char c : password.toCharArray()) {
        if (Character.isDigit(c)) {
            return true;
        }
    }
    return false;
}

public static boolean hasSpecialChar(String password) {
    String specialChars = "!@#$%^&*()-_=+[]{}|;:'\",.<>?/`~";
    for (char c : password.toCharArray()) {
        if (specialChars.indexOf(c) != -1) {
            return true;
        }
    }
    return false;
}

public static String strength(String password) {
```

```
        int score = 0;

        if (hasMinLength(password)) score++;
        if (hasUppercase(password)) score++;
        if (hasDigit(password)) score++;
        if (hasSpecialChar(password)) score++;

       switch (score) {
           case 4:
               return "Strong";
           case 3:
               return "Moderate";
           case 2:
               return "Weak";
           default:
               return "Very Weak";
       }
    }
}
```

CODE : Driver.java

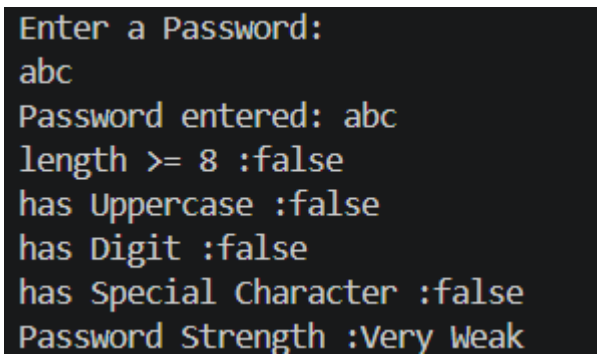
```
public class Driver {
```

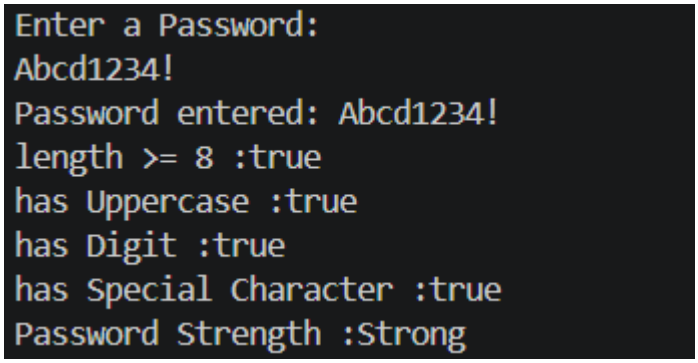
```
    public static void main(String[] args) {


        System.out.println("Enter a Password: ");
        String password = System.console().readLine();
        System.out.println("Password entered: " + password);


    System.out.println("length >= 8 :" + (PasswordChecker.hasMinLength(password)));
    System.out.println("has Uppercase :" + (PasswordChecker.hasUppercase(password)));
    System.out.println("has Digit :" + (PasswordChecker.hasDigit(password)));
    System.out.println("has Special Character :" +
(PasswordChecker.hasSpecialChar(password)));
    System.out.println("Password Strength :" + (PasswordChecker.strength(password)));
    }
}
```

OUTPUT :





PROBLEM-2 : Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts).
(b) Read a keyword from the user.
(c) For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts.
(d) If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder.
(e) Print “Matches: N” and the built report.
Expected output: keyword “hello” over 3 lines (1 malformed) → “Matches: 1” and the matching line.

CODE : ChatFilter.java

```
public class ChatFilter {

    /**
     * Container class to return both the match count and the built report.
     */
    public static class FilterResult {
        private final int matchCount;
```

```
    private final String report;

    public FilterResult(int matchCount, String report) {
        this.matchCount = matchCount;
        this.report = report;
    }

    public int getMatchCount() {
        return matchCount;
    }

    public String getReport() {
        return report;
    }
}

/**
 * Filters log lines based on a case-insensitive keyword match in the message section.
 */
public static FilterResult filterLogs(String[] logs, String keyword) {
    int count = 0;
    StringBuilder sb = new StringBuilder();
    String lowerKeyword = keyword.toLowerCase();
```

```
for (String line : logs) {
    // (c) Split into time, user, and message (at most 3 parts)
    String[] parts = line.split(" ", 3);

    // Skip lines that do not have all 3 parts (malformed lines)
    if (parts.length < 3) {
        continue;
    }

    String time = parts[0];
    String user = parts[1];
    String message = parts[2];

    // (d) Case-insensitive check on the message body
    if (message.toLowerCase().contains(lowerKeyword)) {
        count++;
        sb.append(time)
          .append(" ")
          .append(user)
          .append(": ")
          .append(message)
          .append("\n");
    }
}
```

```
        return new FilterResult(count, sb.toString());
    }
}
```

CODE : Driver.java

```
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        // (a) Sample log lines with 1 malformed line (< 3 parts)
        String[] logs = {
            "10:05 alice Hello there",
            "10:06 System error",      // Malformed line (only 2 parts)
            "10:07 charlie Good morning"
        };

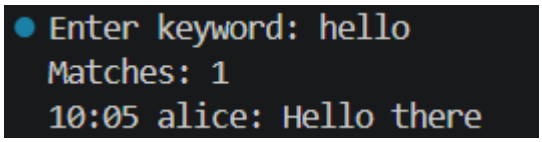
        // (b) Read keyword from user
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter keyword: ");
        String keyword = scanner.nextLine();

        // Process log filtering
        ChatFilter.FilterResult result = ChatFilter.filterLogs(logs, keyword);
```

```
        // (e) Output the results
        System.out.println("Matches: " + result.getMatchCount());
        System.out.print(result.getReport());

        scanner.close();
    }
}
```

OUTPUT :



PROBLEM-3 : Template filler (TemplateFiller.java + Driver.java)

(a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date} is not supplied).
(b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template.
(c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found.
(d) Print the filled template.
Expected output: the unsupplied {date} becomes [?].

CODE : TemplateFiller.java

```
import java.util.regex.Matcher;
```

```
import java.util.regex.Pattern;

public class TemplateFiller {

    public static String fillTemplate(String template, String[] names, String[]
values) {

        Pattern pattern = Pattern.compile("\\{(\\w+)\\}");
        Matcher matcher = pattern.matcher(template);

        StringBuilder result = new StringBuilder();
        int lastEnd = 0;

        while (matcher.find()) {
            result.append(template, lastEnd, matcher.start());

            String placeholder = matcher.group(1);
            String replacement = "[?]";

            for (int i = 0; i < names.length; i++) {
                if (names[i].equals(placeholder)) {
                    replacement = values[i];
                    break;
                }
            }
```

```
            result.append(replacement);
            lastEnd = matcher.end();
        }

        result.append(template, lastEnd, template.length());

        return result.toString();
    }
}
```

CODE : Driver.java

```
import java.util.Scanner;

public class Driver {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter template: ");
        String template = sc.nextLine();

        System.out.print("Enter number of placeholders: ");
```

```
        int n = sc.nextInt();

        String[] names = new String[n];
        String[] values = new String[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter placeholder name: ");
            names[i] = sc.next();

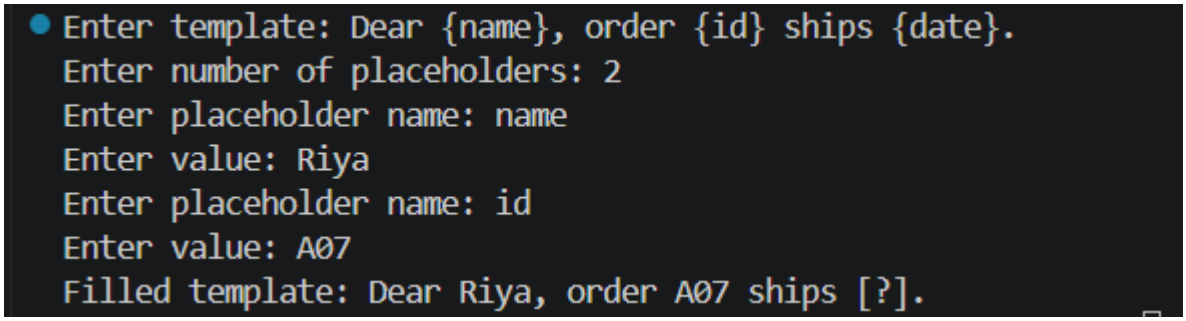
            System.out.print("Enter value: ");
            values[i] = sc.next();
        }

        String result = TemplateFiller.fillTemplate(template, names, values);

        System.out.println("Filled template: " + result);

        sc.close();
    }
}
```

OUTPUT :



Key Questions :

1. What is the difference between String, StringBuilder and StringBuffer, and when is each preferred?

Answer:

- String: Immutable (cannot be changed).
- StringBuilder: Mutable and faster, but not thread-safe.
- StringBuffer: Mutable and thread-safe, but slower than StringBuilder.
- Use String for fixed text, StringBuilder for frequent modifications, and StringBuffer in multi-threaded environments.

2. What does the regular expression [6-9][0-9]{9} match, and why use anchors?

Answer:

- Matches a 10-digit number starting with 6, 7, 8, or 9.
- Anchors (^ and $) ensure the entire input matches the pattern, e.g., ^[6-9][0-9]{9}$.

3. Why should input be validated at the boundary before it is used?

Answer:

- Prevents invalid or malicious data from entering the program.
- Improves security, reliability, and reduces errors later in processing.